Homework 5 Template: Model-Based and Offline RL

CMU 10-703: Deep Reinforcement Learning (Fall 2026)

Use this template to record your answers for Homework 5. Add your answers using LaTeX and then save your document as a PDF to upload to Gradescope. You are required to use this template to submit your answers. **You should not alter this template in any way** other than to insert your solutions. You must submit all 13 pages of this template to Gradescope. Do not remove the instructions page(s). Altering this template or including your solutions outside of the provided boxes can result in your assignment being graded incorrectly.

You should also zip your `code/` directory and upload it to the **separate** Gradescope coding assignment. Remember to mark all teammates on **both** assignment uploads through Gradescope.

Instructions for Specific Problem Types

On this homework, you must fill in blanks for each problem. Please make sure your final answer is fully included in the given space. **Do not change the size of the box provided.** For short answer questions you should **not** include your work in your solution. Only provide an explanation or proof if specifically asked. To include a plot, use `\includegraphics[width=\linewidth]{plot.png}` inside the box, scaled so that it fits.

Fill in the blank: What is the course number? 10703

Problem 0: Collaborators

Enter your team members' names and Andrew IDs in the boxes below. If you worked in a team with fewer than three people, leave the extra boxes blank.

Name 1:	Arnav Goyal	Andrew ID 1:	arnavgoy
Name 2:	Kushal Kejriwal	Andrew ID 2:	kkejriwa
Name 3:	Vineeth Bhat	Andrew ID 3:	vineethb

Collaboration: Detail the work division amongst your group below.

Solution

Kushal did Problem 1, Arnav did Problem 2 and Vineeth did Problem 3

Part I: Model-Based RL

Problem 1: Model-Based RL with PETS (105 pts)

1.1 Model-Based Predictive Control (25 pts)

1.1.1 (10 pts) CEM (without MPC) with ground-truth dynamics.

Success percentage: 94 %

1.1.2 (10 pts) Random sampling with ground-truth dynamics.

Success percentage without MPC: 70 %

Success percentage with MPC: 94 %

How does the performance of random sampling compare to that of CEM?

Solution

Both methods sample actions from the same initial Gaussian distribution, but CEM iteratively updates its sampling distribution using elite trajectories, while random shooting samples all trajectories independently and only selects the best at the end. Thus CEM generally finds better open-loop plans. MPC improves random shooting by replanning after every executed action, correcting errors caused by imperfect plans which explains its higher success rate.

1.1.3 (5 pts) MPC vs. open-loop control: which performs better on what environments, and why? Discuss the pros and cons of MPC.

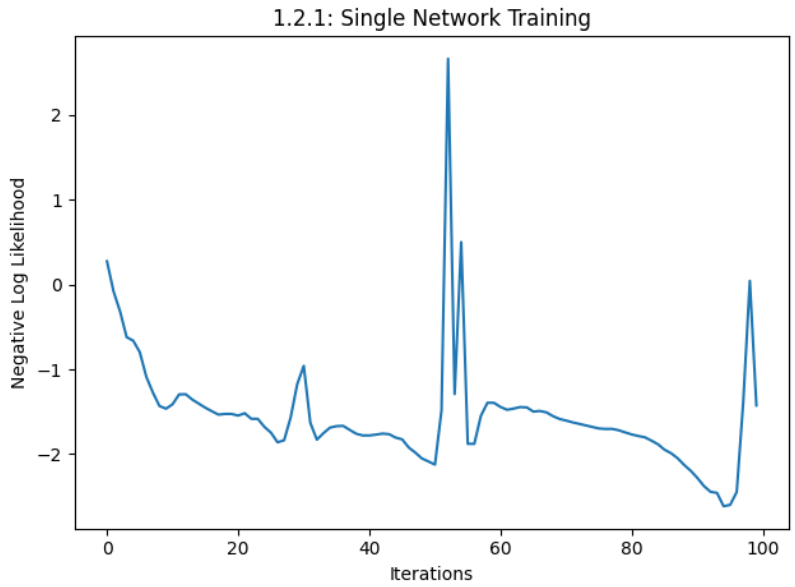
Solution

For Pushing2d-v1, we get 86% and 96 % without and with MPC respectively. For Pushing2dNoisyControl-v1, we get 58 % and 80 % without and with MPC respectively.
MPC is expected to perform better in environments with noisy actions/stochastic dynamics becuase it repeatedly observes the true state and replans.
Open-loop control can be preferable in deterministic environments with accurate dynamics where a planned sequence is already reliable. MPC's main advantage is replanning after every executed action, which allows is to correct errors. MPC's main disadvantages are its computational cost and with a long horizon or expensive optimizer, replanning can be slow.

1.2 Single Probabilistic Network (30 pts)

1.2.1 (4 pts) Training loss plot.

Solution



1.2.2 (10 pts) MPC with random sampling. Success percentage: 30 %

1.2.3 (10 pts) MPC with CEM. Success percentage: 46 %

1.2.4 (6 pts) Random sampling vs. CEM. How did MPC with this model perform in general? When does the derived policy succeed, and when does it fail?

Solution

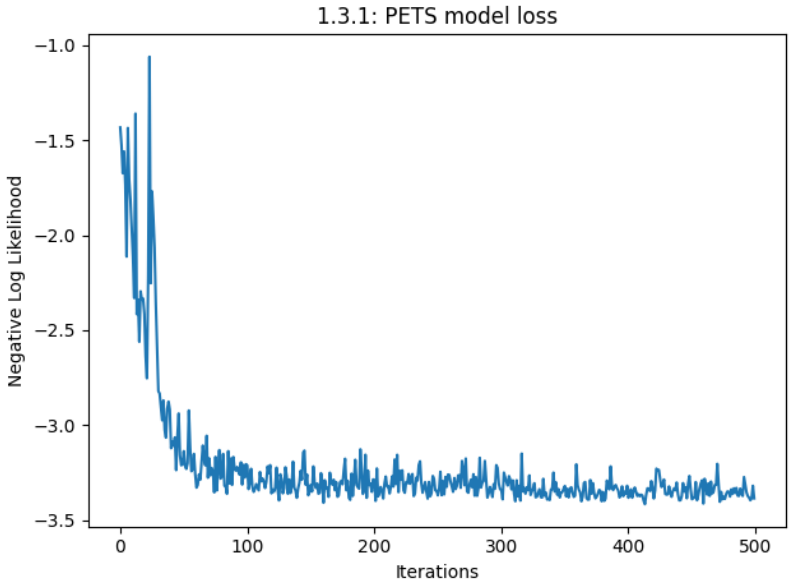
Random shooting achieved an 30% success rate, while CEM achieved 46%. CEM is able to perform better than random sampling because it able to explore the search space in a more effective way, and also updates its parameters based on observed performances, whereas random sampling does not utilise this information. MPC used with CEM seems to perform best for these particular settings, becasue MPC is able to incorporate from the environment, and is able to correct errors caused by imperfect plans

The derived policy is the MPC controller using the learned dynamics model. It succeeds when short-horizon predictions are accurate and fails when model errors compound, especially during pusher-box contact. MPC helps by replanning after each action, but cannot fully correct systematic model errors

1.3 MBRL with PETS (15 pts)

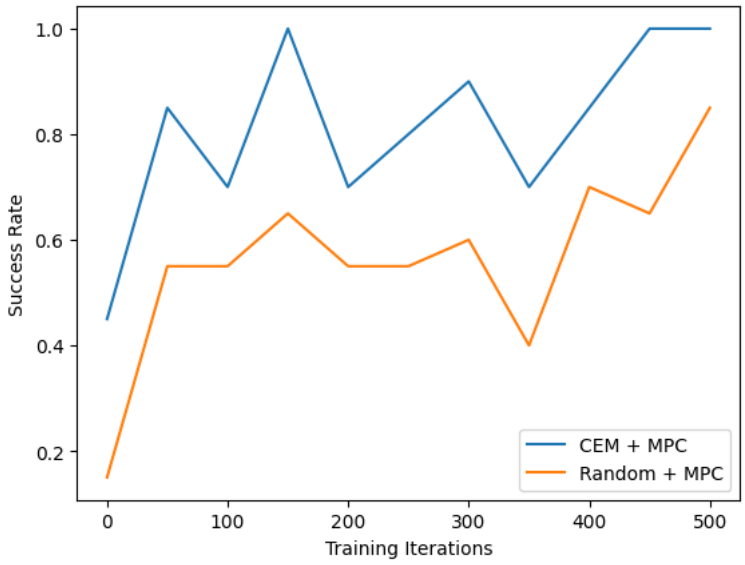
1.3.1 (10 pts) Training loss plot.

Solution



1.3.2 (5 pts) Test success percentage vs. PETS iterations plot.

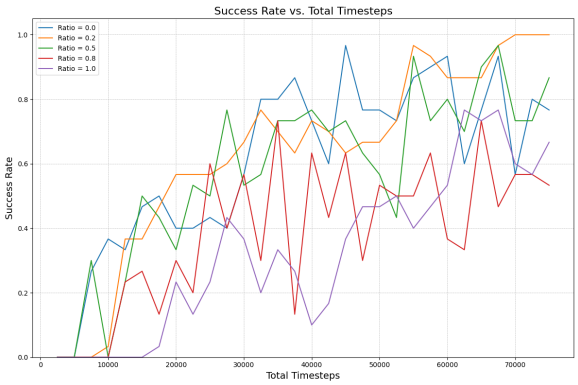
Solution



1.4 Training with Synthetic Data (35 pts)

1.4.1 (10 pts) Ablation over model vs. real data: plot and discussion.

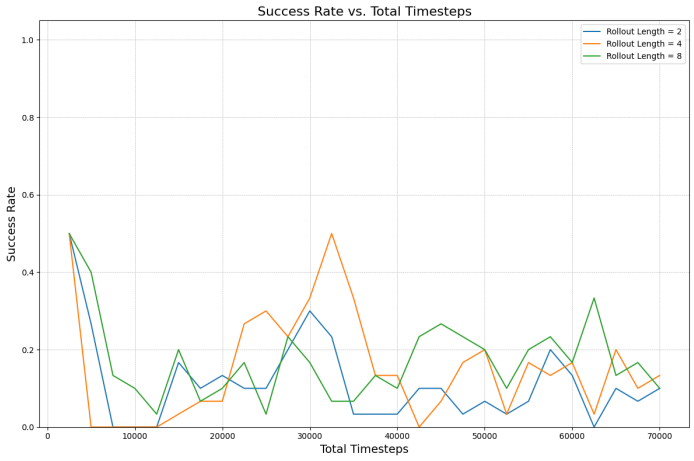
Solution



The results are noisy and do not show a clear monotonic relationship between the synthetic-data ratio and success rate. In general we observe runs with low amount of synthetic data perform reasonably well agsainst runs with high amounts of synthetic data. Overall, these results suggest that a small or moderate amount of synthetic data can be useful, but relying entirely on model-generated data can lead to unstable performance. Real data anchors TD3 to actual environment behavior, while synthetic data increases the amount of experience available for training.

1.4.2 (10 pts) Ablation over model rollout lengths: plot and discussion.

Solution



The success-rate curves are highly variable and show no clear trend, and all of them result in poor success rates. In general we expect shorter rollouts to perform better because they limit compounding errors. Longer rollouts provide more synthetic transitions per starting state, but they also make TD3 more dependent on model predictions farther from the real data distribution.

1.4.3 (10 pts) Model error vs. rollout length: plot and discussion.

Solution



Unlike the success-rate plots, the model-error plot shows a clear pattern. The mean L2 prediction error increases steadily with rollout step, from approximately 0.15 at the first step to approximately 0.5 by step 15. This confirms that model errors compound as predictions are fed back into the model. The shaded area also increases overall, indicating greater variance at longer rollouts which is also a consequence of compounding errors

1.4.4 (5 pts) Under which scenarios would you prefer MBRL to policy gradient methods?

Solution

MBRL is preffered when environment interaction is expensive or inefficient, because a learned dynamics model can generate many imagined or hypothetical transitions from each real interaction. MBRL can also be useful when the system has reasonably predictable/determinisitic dynamics and planning can exploit that structure. Policy-gradient methods may be preferable when the dynamics are highly complex or difficult to model accurately. They can learn directly from real experience without suffering from long horizon errors (as seen in 1.4.2).

Part II: Offline RL

Problem 2: TD3 + Behavior Regularization (50 pts)

2.2 Running TD3 without Any Offline Learning Modifications

Learning curves (evaluation returns and Q-values), and: what insights can we draw from evaluation returns being low but predicted Q-values being high?

Solution

2.3 Behavior Regularized TD3

Learning curves (evaluation returns and Q-values) with $\alpha_{\text{reg}} = 10$.

Solution

2.4 Conceptual Questions about Picking α_{reg}

1. For which kinds of datasets will we expect larger values of α_{reg} to work better? For which datasets will we expect lower values to work better?

   Solution

2. Given a completed run with $\alpha_{\text{reg}} = 1.0$, how would you choose a reasonable value of α_{reg} if you could only run a single more run?

Solution

Problem 3: Conceptual Questions on Offline RL (50 pts)

(i) Select all that apply and justify (see the handout for the three methods and statements 1–3).

Solution

Selected: 1 and 3. 1 (*True*). With high coverage, $\hat{\pi}_\beta(a|s)$ is not ~ 0 for truly good actions, so CQL's value penalty $\alpha(\mu/\hat{\pi}_\beta - 1)$ stays small, so the actor can still commit to good actions. KL instead acts on the policy itself, so if π_β is mediocre despite good support, the learnt policy ($\pi \propto \pi_\beta e^{Q/\beta}$) can only shift a limited amount of probability mass toward truly good actions, set by the constraint strength. It also needs an explicitly fitted $\hat{\pi}_\beta$, which is hard to fit on broad, multi-policy data, whereas CQL only uses dataset samples. 2 (*False*). Less conservative is not always better since the reward penalty is only as good as $u(s, a)$, which is notoriously hard to get right. Further, CQL's penalty $\alpha(\mu/\hat{\pi}_\beta - 1)$ also propagates through backups, but it vanishes when $\mu = \hat{\pi}_\beta$ and is a bonus on actions the data favors more than π, so CQL only lower-bounds the *expected* policy value, a tighter bound than an uncertainty penalty's pointwise bound on $Q(s, a)$. 3 (*True*). Policy constraints keep π near π_β even where the data supports better actions while CQL drops the explicit constraint and only penalizes Q-values of actions over-represented under π relative to the data (also explained in 1).

(ii) Describe a scenario where distribution constraint methods would be preferable to support constraints.

Solution

Consider a scenario where we have a small dataset of actions from an expert behavior policy along with a minuscule number of sub-optimal/random actions. Since these random actions are technically in support, a support constraint would let the policy choose them freely. Then, if these random actions were poor/sub-optimal, we wouldn't know for sure since we only have a small number of transitions including them, and the $\hat{Q}$ estimates for them could also be high variance. So, if by chance accidentally the few transitions containing these poor actions happened to have good outcomes, $\hat{Q}$ could overestimate them. A support constraint lets the policy put all its mass on such an action whereas a distribution constraint keeps π close to the actions π_β took frequently, where $\hat{Q}$ is well estimated, which in our example is near-expert behavior. Additionally, if diversity is an explicit goal beyond return (e.g. for exploration during later online fine-tuning), a mode-covering distribution constraint such as forward KL $D_{\mathrm{KL}}(\pi_\beta \,\|\, \pi)$ preserves the multimodality of π_β, whereas a support constraint allows the policy to collapse onto a single mode.

(iii) Select all that apply and justify (see the handout for statements 1–3).

Solution

Selected: 2 only. 1 (*False*). Model-based methods only help if the learned dynamics $\hat{T}$ is accurate where the policy rolls out. When you have small datasets or long rollouts, model errors compound with rollout length and the policy ends up exploiting them, so a model-free method such as CQL, which never relies on $\hat{T}$, can do better. So, it is not true for all datasets. 2 (*True*). This is because with narrow data, $\hat{T}$ is accurate only in a thin region around π_β's state-action distribution. Optimizing inside the model, an unconstrained policy quickly rolls out of this region and then model errors become large and get exploited. A distributional constraint keeps π, and thereby the model rollouts, close to π_β, where $\hat{T}$ can be trusted and won't get exploited as much. 3 (*False*). With broad, high-coverage data, $\hat{T}$ is accurate over most of the space, so it is likely to generalize well and won't get exploited easily as in (2). The massive advantage of the model is then generalizing to find behavior much better than π_β. A distributional constraint pulls π toward π_β's (possibly mediocre) action frequencies regardless of coverage, thereby not using this advantage.

(iv) Why can offline RL perform better than imitation learning? Give a dataset where offline RL

should outperform imitation, and one where imitation should outperform offline RL.

Solution

Imitation copies π_β and ignores rewards so bad trajectories & mistakes are also learnt. Offline RL uses rewards and Bellman backups, so it can discard bad actions and *stitch* good segments of different trajectories into a policy better than any trajectory in the data.

Offline RL > imitation: lots of mixed-quality or noisy data with good coverage, such as suboptimal trajectories where none reaches the goal but pieces of several do. BC copies the mediocre average, while offline RL can stitch together the good parts. Offline RL can still win on sparse-reward tasks because of stitching, even though sparse rewards make value learning harder.

Imitation > offline RL: a small set of clean expert demonstrations with narrow coverage. BC is already near-optimal, so there is nothing to improve on, while offline RL must estimate $\hat{Q}$ from little, narrow data. Its value errors and conservatism tuning can start to hurt, and especially so when nearly every state is "critical" (one wrong action is catastrophic, for instance walking along a cliff with a humanoid).

Feedback

Feedback: You can help the course staff improve the course for future semesters by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

Solution

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade. Please average your answer over all the members of your team.

Solution

Alone	
With teammates	
With other classmates	
At office hours	